

# Reinforcement Learning

Nysret Musliu

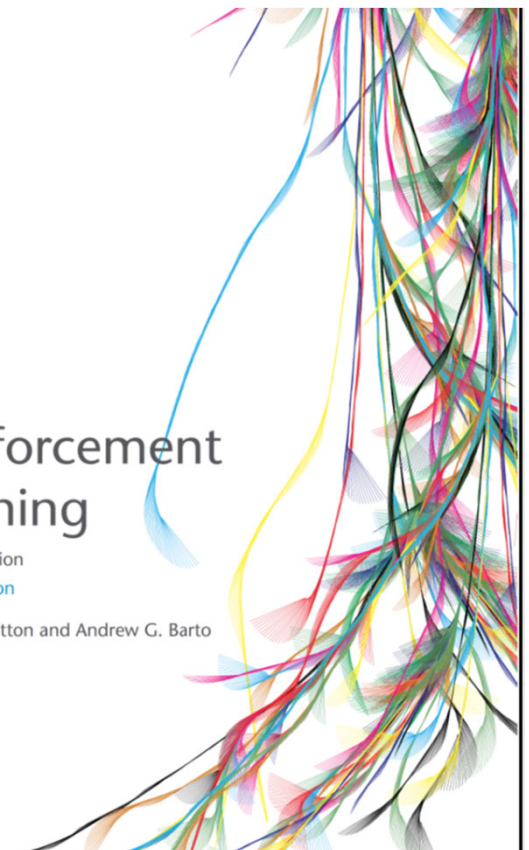
## Reinforcement Learning

An Introduction  
second edition

Richard S. Sutton and Andrew G. Barto

<http://incompleteideas.net/book/RLbook2020.pdf>

- This lecture: Chapter 1- 3 (pages 1- 71) and some parts of chapters 4,5,6
- Slides are based on these chapters

The cover of the book 'Reinforcement Learning: An Introduction, second edition' by Richard S. Sutton and Andrew G. Barto. The cover features a white background with a dense, colorful, abstract pattern of lines and shapes on the right side, resembling a stylized tree or a complex network. The title 'Reinforcement Learning' is in a large, black, sans-serif font. Below it, 'An Introduction' is in a smaller, black, sans-serif font, and 'second edition' is in a smaller, blue, sans-serif font. At the bottom, the authors' names 'Richard S. Sutton and Andrew G. Barto' are in a small, black, sans-serif font.

## Reinforcement Learning

An Introduction  
second edition

Richard S. Sutton and Andrew G. Barto

- Learning by interacting with the environment
- “Reinforcement learning is learning what to do—how to map situations to actions—so as to maximize a numerical reward signal.”  
(Sutton and Barto, RL Book)
- Discover which actions to take by trail-and-error
  - Immediate reward
  - Delayed rewards
- An agent must be able to
  - sense the state of its environment
  - take actions that affect the state

Different from supervised learning

- Learning from a set of training instances provided by supervisor
- In RL an agent must be able to learn from its own experience

Different from unsupervised learning

- Finding structure in unlabeled data
- Reinforcement learning is trying to maximize a reward signal instead of trying to find hidden structure

Reinforcement learning can be considered as a third machine learning paradigm

- Trade-off between exploration and exploitation
- A reinforcement learning agent must prefer actions that give good rewards (exploitation)
- Try also actions that were not selected before to discover good actions (exploration)
- Uncertain environment
- Taking into account delayed consequences of actions
- The effects of actions cannot be fully predicted

## **Policy:**

- Agent's way of behaving at a given time
- A mapping from perceived states of the environment to actions to be taken

## **Reward signal:**

- Defines the goal of a reinforcement learning problem
- The environment sends a reward to agent
- The objective is to maximize the total reward over the long run

## **Value function:**

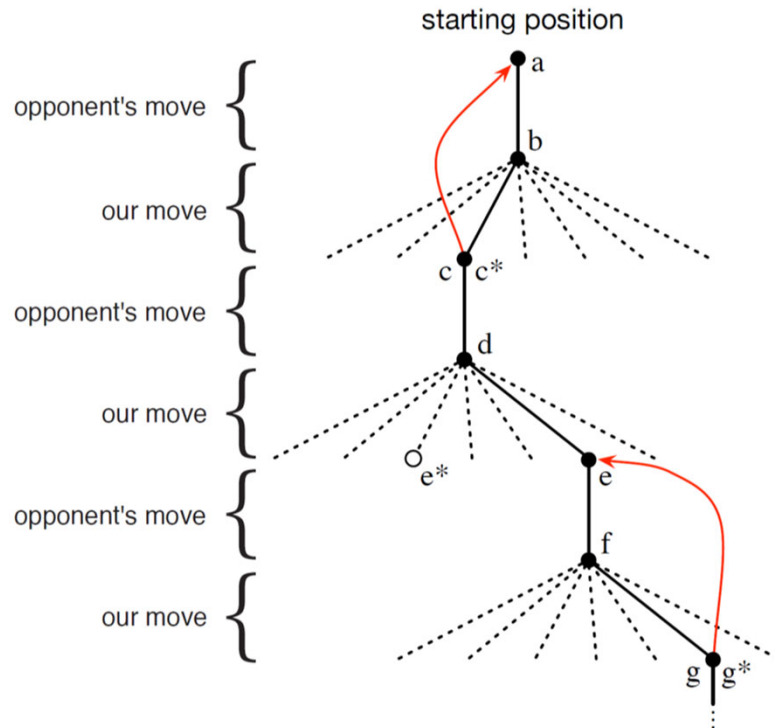
- Specifies what is good in the long run
- The total amount of reward an agent can expect to accumulate over the future

## **Model of the environment:**

- Mimics the behavior of the environment
- .....



# Example: Tic-Tac-Toe



$S_t$ : the state before the greedy move

$S_{t+1}$ : the state after the move

The update to the estimated value of  $S_t$ :

$$V(S_t) \leftarrow V(S_t) + \alpha [V(S_{t+1}) - V(S_t)].$$

Alfa is the state-size parameter



- State and action spaces are small
- Methods can often find optimal value function and the optimal policy
- Different from approximate methods which only find approximate solutions, but can be applied effectively to larger problems

## Topics:

Bandit problems

Finite Markov decision processes

Dynamic programming

Monte Carlo methods

Temporal difference learning

# A k-armed Bandit Problem



Wikipedia

# A k-armed Bandit Problem

.....

- Select among  $k$  different actions
- Each action receives a numerical reward
  - A stationary probability distribution that depends on the action you selected
- The value of an arbitrary action  $a$  is the expected reward given that  $a$  is selected:

$$q_*(a) \doteq \mathbb{E}[R_t \mid A_t = a]$$

$A_t$  -> action selected on time step  $t$

$R_t$  - > corresponding reward

- Maximize the expected total reward over some time period

- We denote the estimated value of action  $a$  at time step  $t$  as  $Q_t(a)$
- We would like  $Q_t(a)$  to be close to  $q_*(a)$
- At any time step there is at least one action whose estimated value is greatest
- Selection of a greedy action
  - Exploitation
- Selection of non greedy action
  - Exploration
- Balance between exploitation and exploration is important

- Estimating the values of actions

$$Q_t(a) \doteq \frac{\text{sum of rewards when } a \text{ taken prior to } t}{\text{number of times } a \text{ taken prior to } t} = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbb{1}_{A_i=a}},$$

- If the denominator is zero,  $Q_t(a)$  takes a default value (e.g. 0)
- Sample-average method
- Greedy action selection

$$A_t \doteq \operatorname{argmax}_a Q_t(a)$$

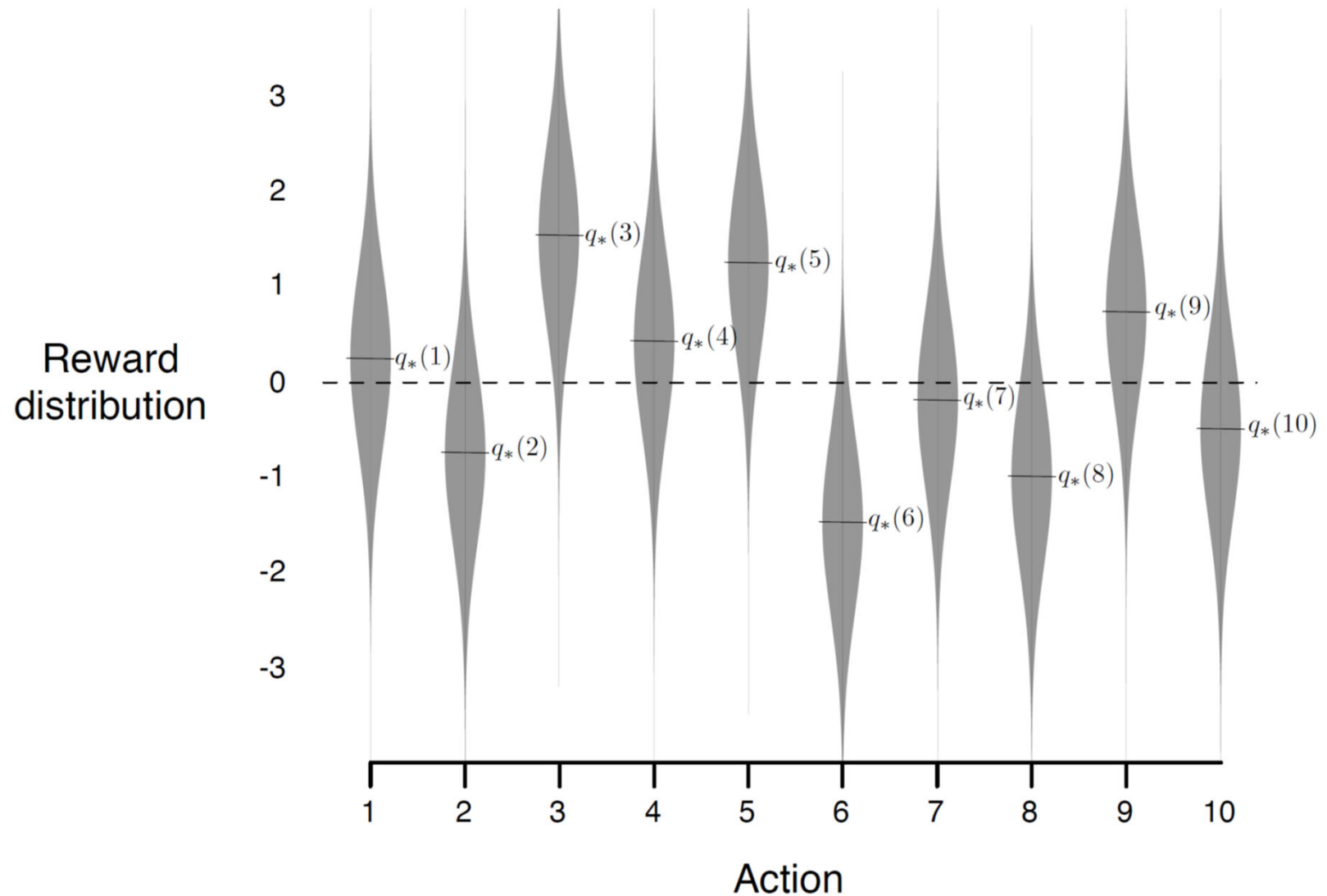
- Greedy action selection does not try inferior actions to see if they might really be better
- With small probability  $\epsilon$  select randomly from among all the actions with equal probability, independently of the action-value estimates
- With probability  $1-\epsilon$  select one of the actions with the highest estimated value

# The 10-armed Testbed

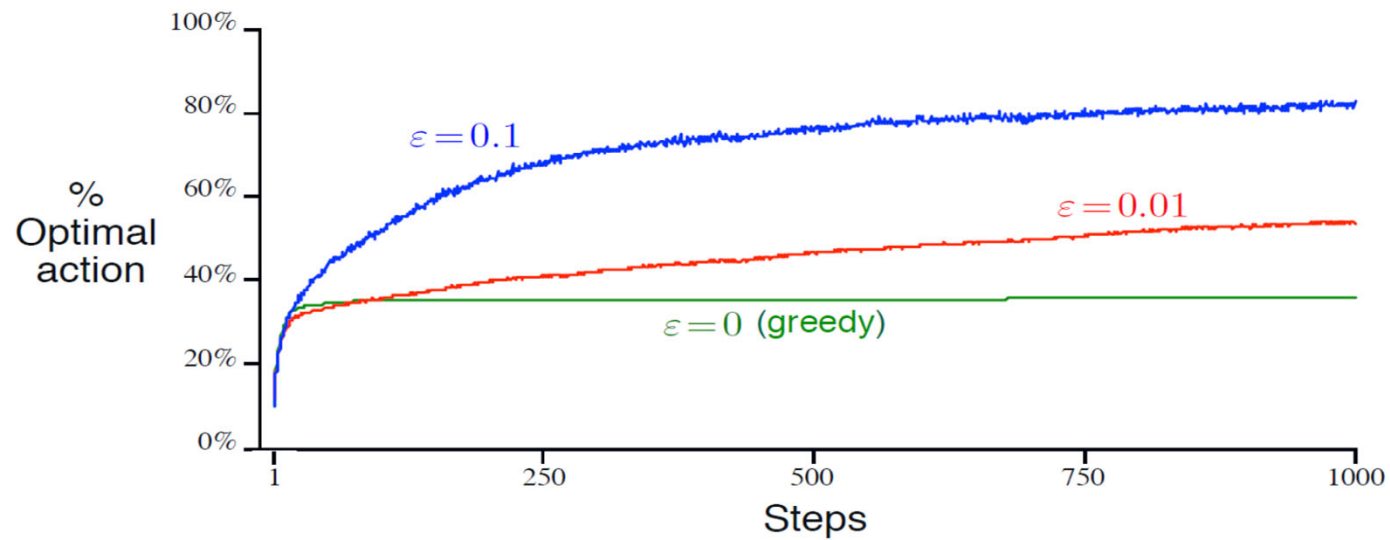
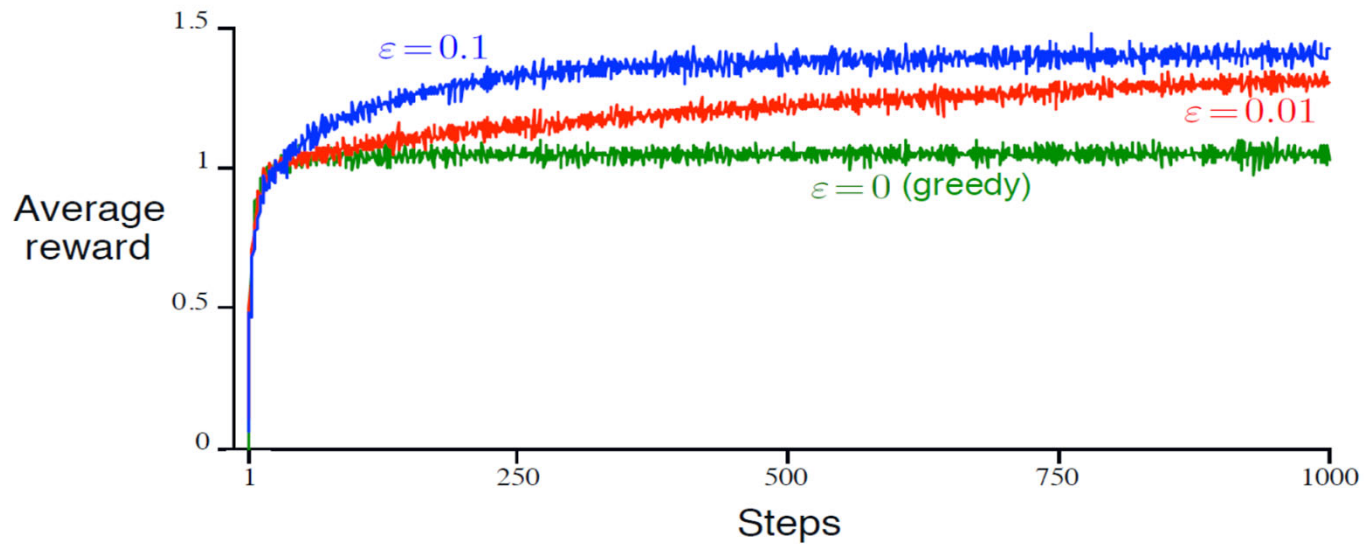
---

- A set of 2000 randomly generated k-armed bandit problems
- $k = 10$
- The  $q_*(a)$  were selected according to a normal (Gaussian) distribution with mean 0 and variance 1
- The actual rewards were selected according to a mean  $q_*(a)$  unit variance normal distribution
- When a learning method applied to that problem selected action  $A_t$  at time step  $t$ , the actual reward,  $R_t$ , was selected from a normal distribution with mean  $q_*(A_t)$  and variance 1

# The 10-armed Testbed: Distributions







# Incremental Implementation

.....

- Concentrate on a single action:  $R_i$  is the reward received after the  $i^{\text{th}}$  selection of this action
- $Q_n$  denote the estimate of its action value after it has been selected  $n - 1$  times,

$$Q_n \doteq \frac{R_1 + R_2 + \dots + R_{n-1}}{n - 1}$$

$$\begin{aligned} Q_{n+1} &= \frac{1}{n} \sum_{i=1}^n R_i \\ &= \frac{1}{n} \left( R_n + \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left( R_n + (n-1) \frac{1}{n-1} \sum_{i=1}^{n-1} R_i \right) \\ &= \frac{1}{n} \left( R_n + (n-1) Q_n \right) \\ &= \frac{1}{n} \left( R_n + n Q_n - Q_n \right) \\ &= Q_n + \frac{1}{n} [R_n - Q_n], \end{aligned}$$

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \left[ \text{Target} - \text{OldEstimate} \right]$$

.....

## A simple bandit algorithm

Initialize, for  $a = 1$  to  $k$ :

$$Q(a) \leftarrow 0$$

$$N(a) \leftarrow 0$$

Loop forever:

$$A \leftarrow \begin{cases} \operatorname{argmax}_a Q(a) & \text{with probability } 1 - \varepsilon \\ \text{a random action} & \text{with probability } \varepsilon \end{cases} \quad (\text{breaking ties randomly})$$

$$R \leftarrow \text{bandit}(A)$$

$$N(A) \leftarrow N(A) + 1$$

$$Q(A) \leftarrow Q(A) + \frac{1}{N(A)} [R - Q(A)]$$

# Non-stationary bandit problem

- Averaging methods are appropriate for stationary bandit problems (reward probabilities do not change over time)
- Reinforcement learning problems are often non-stationary. Give more weights to recent rewards than to long-past rewards

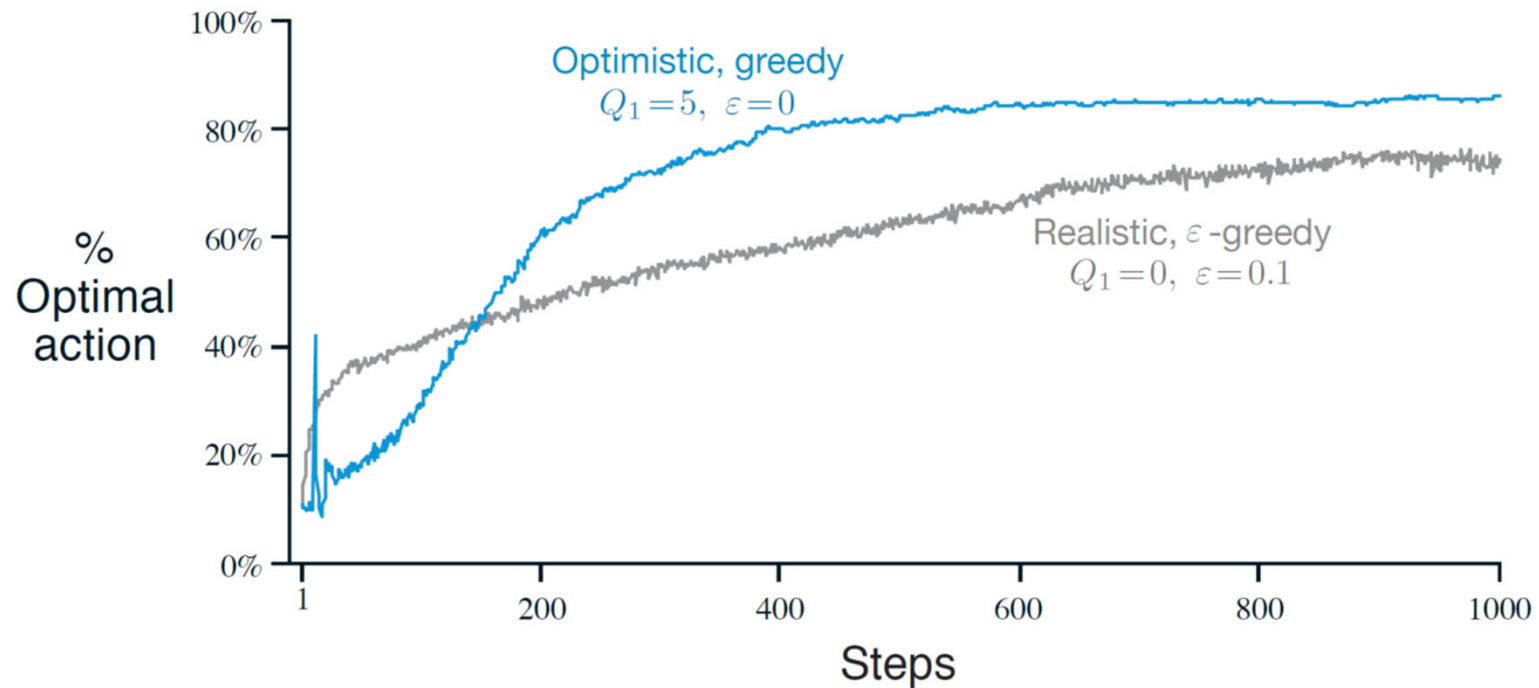
$$Q_{n+1} \doteq Q_n + \alpha [R_n - Q_n] \quad \alpha \in (0, 1]$$

$$\begin{aligned} Q_{n+1} &= Q_n + \alpha [R_n - Q_n] \\ &= \alpha R_n + (1 - \alpha) Q_n \\ &= \alpha R_n + (1 - \alpha) [\alpha R_{n-1} + (1 - \alpha) Q_{n-1}] \\ &= \alpha R_n + (1 - \alpha) \alpha R_{n-1} + (1 - \alpha)^2 Q_{n-1} \\ &= \alpha R_n + (1 - \alpha) \alpha R_{n-1} + (1 - \alpha)^2 \alpha R_{n-2} + \\ &\quad \dots + (1 - \alpha)^{n-1} \alpha R_1 + (1 - \alpha)^n Q_1 \\ &= (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i. \end{aligned}$$

- Initial action values can also be used to encourage exploration
- Set larger initial action values
- The reward for actions is less than the starting estimates
- The learner switches to other actions
- All actions are tried several times before the value estimates converge
- Although greedy actions are selected, an exploration is encouraged
- Not good for non-stationary problems

# Example

- 10-armed bandit testbed of a greedy method using  $Q_1(a) = +5$ , for all actions



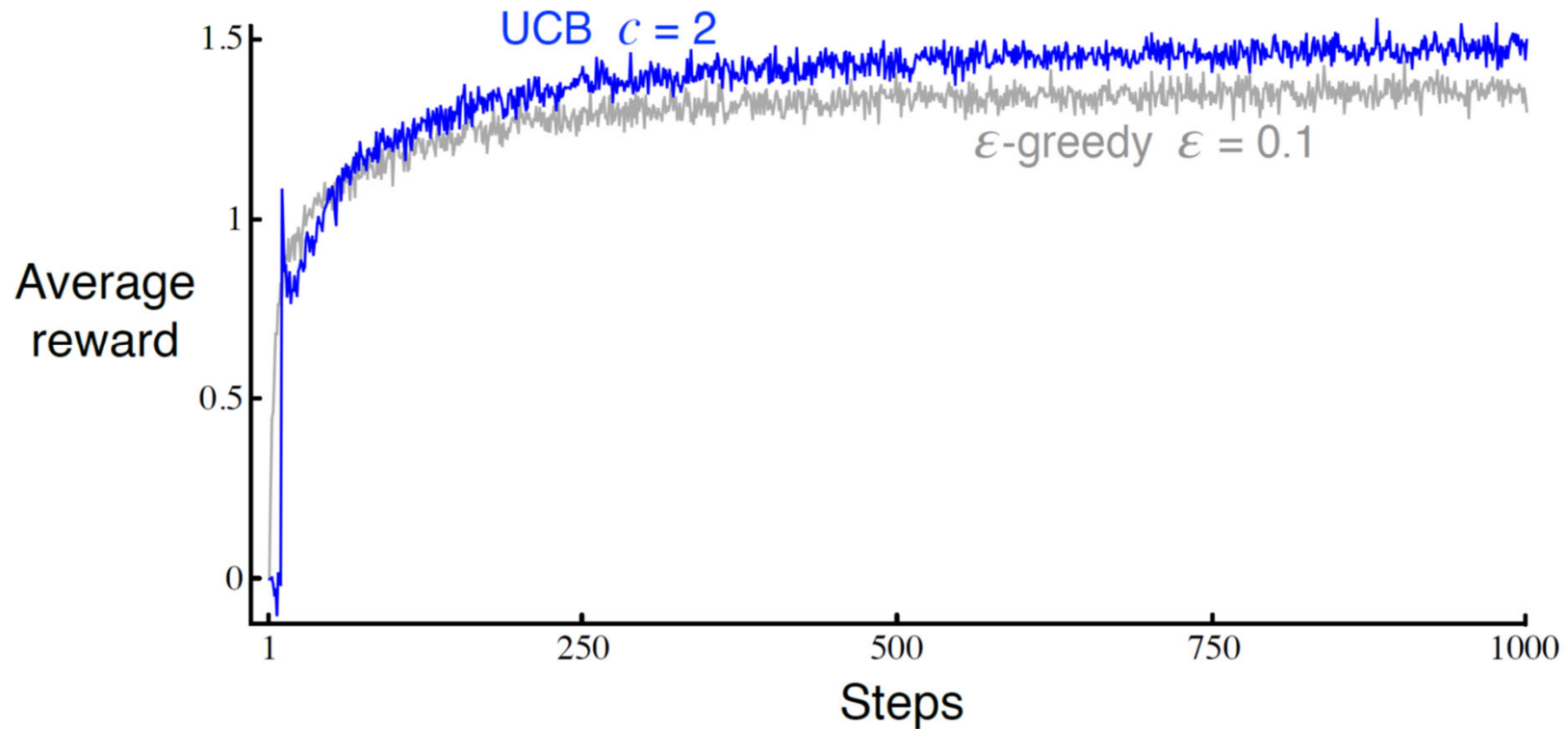
- $\varepsilon$ - greedy also tries non-greedy actions
- No preference is given to selected actions
- Select according to how close their estimates are to being maximal and the uncertainties in those estimates
- One possibility is to select actions like below

$$A_t \doteq \arg \max_a \left[ Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right]$$

- $\ln t$  : the natural logarithm of  $t$
- $N_t(a)$ : the number of times that action  $a$  has been selected prior to  $t$
- $c > 0$ : controls the degree of exploration

# Example

.....



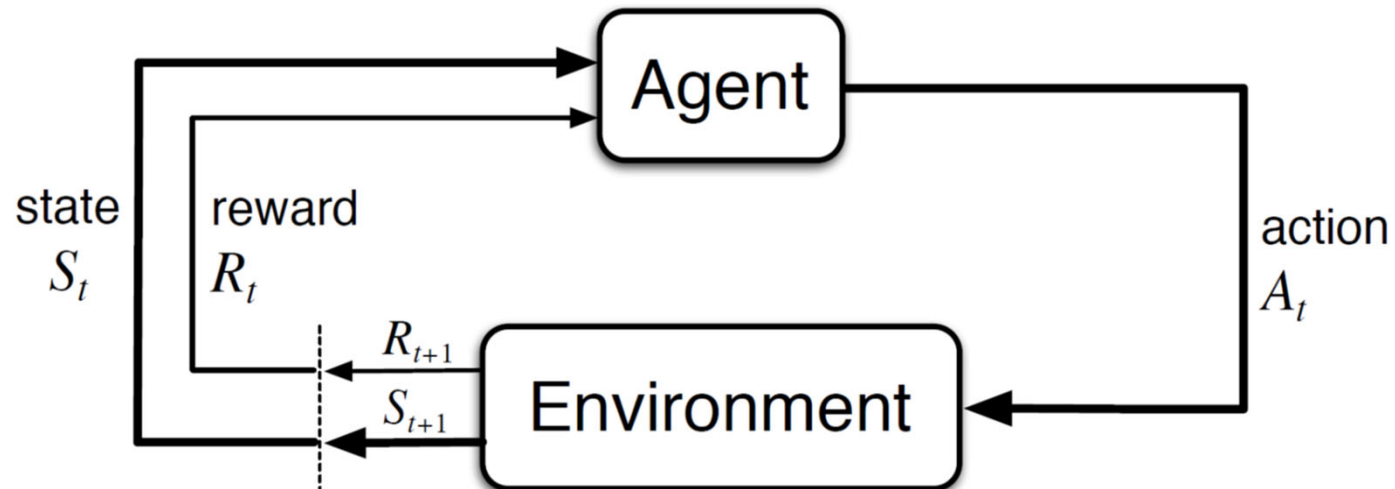


# Markov Decision Processes

- We consider finite Markov decision processes (MDPs)
- Feedback, as in bandits, but also selection of different actions in different situations
- Actions influence also subsequent situations and future rewards
- In MDPs we estimate the value  $q_*(s, a)$  of each action  $a$  in each state  $s$
- The agent and environment interact at each of a sequence of discrete time steps,  $t = 0, 1, 2, 3,$
- At each time step  $t$   $S_t \in \mathcal{S}$

- Selects an action  $A_t \in \mathcal{A}(s)$
- As a consequence of its action, the agent receives a numerical reward  $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ .

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$$



**Figure 3.1:** The agent–environment interaction in a Markov decision process.

- Random variables  $R_t$  and  $S_t$  have well defined discrete probability distributions dependent only on the preceding state and action

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\}.$$

for all  $s', s \in \mathcal{S}$ ,  $r \in \mathcal{R}$ , and  $a \in \mathcal{A}(s)$

$$p : \mathcal{S} \times \mathcal{R} \times \mathcal{S} \times \mathcal{A} \rightarrow [0, 1]$$

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r | s, a) = 1, \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s)$$

$$p(s' | s, a) \doteq \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r | s, a)$$

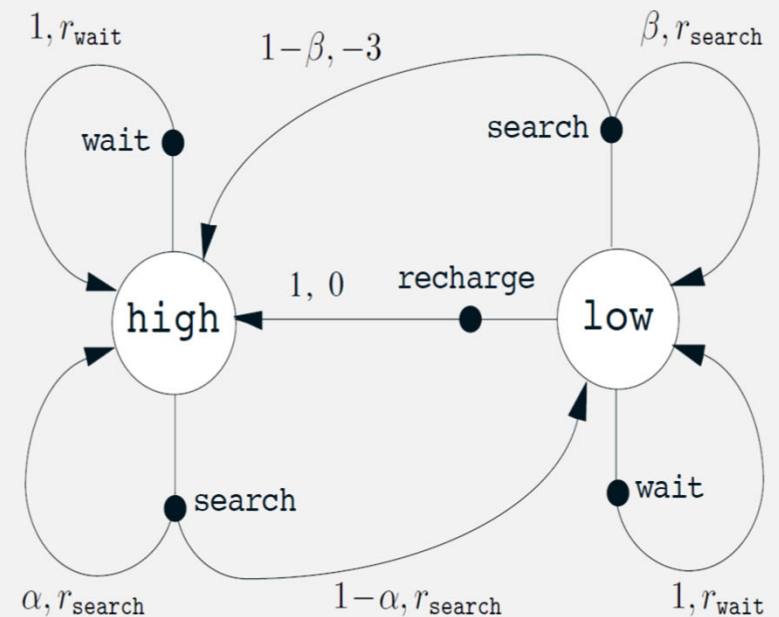
$$r(s, a) \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r | s, a)$$

$$r(s, a, s') \doteq \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r | s, a)}{p(s' | s, a)}$$

# Example: Recycling Robot

- Collecting empty soda cans in an office environment

| $s$  | $a$      | $s'$ | $p(s'   s, a)$ | $r(s, a, s')$       |
|------|----------|------|----------------|---------------------|
| high | search   | high | $\alpha$       | $r_{\text{search}}$ |
| high | search   | low  | $1 - \alpha$   | $r_{\text{search}}$ |
| low  | search   | high | $1 - \beta$    | $-3$                |
| low  | search   | low  | $\beta$        | $r_{\text{search}}$ |
| high | wait     | high | 1              | $r_{\text{wait}}$   |
| high | wait     | low  | 0              | -                   |
| low  | wait     | high | 0              | -                   |
| low  | wait     | low  | 1              | $r_{\text{wait}}$   |
| low  | recharge | high | 1              | 0                   |
| low  | recharge | low  | 0              | -                   |



Sequence of rewards

$$R_{t+1}, R_{t+2}, R_{t+3}, \dots,$$

Maximize the expected return

$$G_t \doteq R_{t+1} + R_{t+2} + R_{t+3} + \dots + R_T,$$

Applications with final time step

Agent–environment interaction breaks into episodes

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

$$0 \leq \gamma \leq 1$$

$\gamma$  -> discount rate

$\gamma = 0$  -> maximizing immediate rewards

$\gamma$  approaches 1 -> future rewards are taken into account more strongly

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \\ &= R_{t+1} + \gamma (R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \cdots) \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$



- The sum is finite if the reward is nonzero and constant
- For example, if the reward is a constant +1, then the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1 - \gamma}.$$

- A policy is a mapping from states to probabilities of selecting each possible action
- $\pi(a|s)$ : probability of selecting action  $a$  in state  $s$

Value function of a state  $s$  under a policy  $\pi$ :

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s\right], \text{ for all } s \in \mathcal{S}$$

The value of taking action  $a$  in state  $s$  under a policy  $\pi$ :

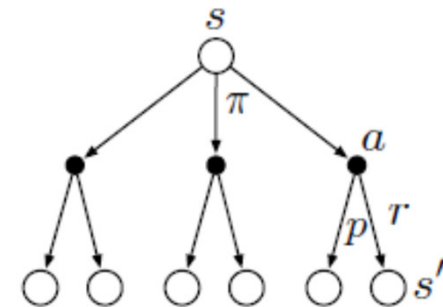
$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a\right]$$

# Bellman equation

It expresses a relationship between the value of a state and the values of its successor states

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) \left[ r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1} = s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) \left[ r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S}, \end{aligned}$$

The equation averages over all the possibilities, weighting each by its probability of occurring



Backup diagram for  $v_{\pi}$

# Optimal Policies and Optimal Value Functions

.....

- RL: find a policy that achieves a lot of reward over the long run
- Optimal policy  $\pi_*$
- Optimal policies share the same:

Optimal state-value function

$$v_*(s) \doteq \max_{\pi} v_{\pi}(s)$$

Optimal action-value function

$$q_*(s, a) \doteq \max_{\pi} q_{\pi}(s, a)$$

$$q_*(s, a) = \mathbb{E}[R_{t+1} + \gamma v_*(S_{t+1}) \mid S_t = s, A_t = a]$$

- DP can be used to compute the value functions
- Finding optimal policies based on Bellman equations
- The dynamic of system needed
- Computationally expensive

- Based on experience: sample sequences of states, actions, and rewards
- No prior knowledge of the environment's dynamics required
- Value estimates and policies are changed only on the completion of an episode
- Average sample returns
- Similar to the bandit methods
- Multiple states: each similar to a different bandit problem
- Different bandit problems are interrelated
  - The return after taking an action in one state depends on the actions taken in later states in the same episode

- Learn the state-value function for a given policy
- Average the returns observed after visits to that state
- With sufficient returns the average should converge to the expected value
- We have given a policy and a set of episodes
- A visit to state  $s$ : each occurrence of state  $s$  in an episode
- $s$  may be visited multiple times in the same episode
- The first-visit Monte Carlo method estimates  $v(s)$  as the average of the returns following first visits to  $s$

## First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy  $\pi$  to be evaluated

Initialize:

$V(s) \in \mathbb{R}$ , arbitrarily, for all  $s \in \mathcal{S}$

$Returns(s) \leftarrow$  an empty list, for all  $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following  $\pi$ :  $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless  $S_t$  appears in  $S_0, S_1, \dots, S_{t-1}$ :

Append  $G$  to  $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

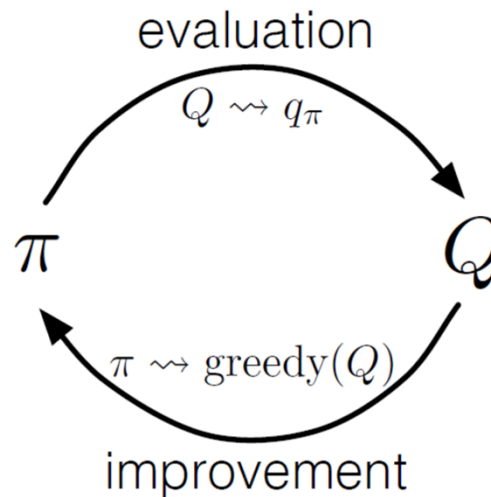
- First-visit MC converges to  $v_\pi(s)$  as the number of visits  $s$  goes to infinity



- With a model, state values are sufficient to determine a policy
- Without a model, state values alone are not sufficient
- Estimate the value of each action
- Policy evaluation problem for action values
  - Estimate  $q_{\pi}(s, a)$ : the expected return when starting in state  $s$ , taking action  $a$ , and thereafter following policy  $\pi$ .
- Similar to the Monte Carlo methods for state values
- A state–action pair  $s, a$  is said to be visited in an episode if ever the state  $s$  is visited and action  $a$  is taken in it
- The every-visit MC method estimates the value of a state–action pair as the average of the returns that have followed all the visits to it

- State–action pairs may never be visited
- Exploring starts
  - Specifying that the episodes start in a state–action pair
  - Every pair has a nonzero probability of being selected as the start
- Alternative: consider only policies that are stochastic with a nonzero probability of selecting all actions in each state

- Approximate optimal policies
- The value function is repeatedly altered to more closely approximate the value function for the current policy
- The policy is repeatedly improved with respect to the current value function



$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \dots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

- Make the policy greedy with respect to the current action-value function
- For any action-value function  $q$ , the corresponding greedy policy is the one that, for each  $s$ , deterministically chooses an action with maximal action-value:  $\pi(s) \doteq \arg \max_a q(s, a)$

$$\begin{aligned} q_{\pi_k}(s, \pi_{k+1}(s)) &= q_{\pi_k}(s, \arg \max_a q_{\pi_k}(s, a)) \\ &= \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}(s, \pi_k(s)) \\ &\geq v_{\pi_k}(s). \end{aligned}$$

## Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:

$\pi(s) \in \mathcal{A}(s)$  (arbitrarily), for all  $s \in \mathcal{S}$

$Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

Loop forever (for each episode):

Choose  $S_0 \in \mathcal{S}$ ,  $A_0 \in \mathcal{A}(S_0)$  randomly such that all pairs have probability  $> 0$

Generate an episode from  $S_0, A_0$ , following  $\pi$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair  $S_t, A_t$  appears in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$ :

Append  $G$  to  $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

## On-policy first-visit MC control (for $\varepsilon$ -soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small  $\varepsilon > 0$

Initialize:

$\pi \leftarrow$  an arbitrary  $\varepsilon$ -soft policy

$Q(s, a) \in \mathbb{R}$  (arbitrarily), for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$  empty list, for all  $s \in \mathcal{S}$ ,  $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following  $\pi$ :  $S_0, A_0, R_1, \dots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode,  $t = T-1, T-2, \dots, 0$ :

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair  $S_t, A_t$  appears in  $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$ :

Append  $G$  to  $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \operatorname{argmax}_a Q(S_t, a)$  (with ties broken arbitrarily)

For all  $a \in \mathcal{A}(S_t)$ :

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

# Optional Slides

- Central idea of reinforcement learning
- TD methods can learn directly from raw experience without a model of the environment's dynamics

## TD Prediction

- A simple every-visit Monte Carlo method

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

- Monte Carlo methods must wait until the end of the episode
- TD methods need to wait only until the next time step

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



## Tabular TD(0) for estimating $v_\pi$

Input: the policy  $\pi$  to be evaluated

Algorithm parameter: step size  $\alpha \in (0, 1]$

Initialize  $V(s)$ , for all  $s \in \mathcal{S}^+$ , arbitrarily except that  $V(\text{terminal}) = 0$

Loop for each episode:

    Initialize  $S$

    Loop for each step of episode:

$A \leftarrow$  action given by  $\pi$  for  $S$

        Take action  $A$ , observe  $R, S'$

$V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$

$S \leftarrow S'$

    until  $S$  is terminal

# Sarsa: On-policy TD Control

---

- Learn an action-value function
- Transitions from state–action pair to state–action pair

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[ R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t) \right]$$

## Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+$ ,  $a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

    Initialize  $S$

    Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

    Loop for each step of episode:

        Take action  $A$ , observe  $R, S'$

        Choose  $A'$  from  $S'$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

$Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$

$S \leftarrow S'; A \leftarrow A';$

    until  $S$  is terminal

# Q-learning: Off-policy TD Control

.....

## Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size  $\alpha \in (0, 1]$ , small  $\varepsilon > 0$

Initialize  $Q(s, a)$ , for all  $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$ , arbitrarily except that  $Q(\text{terminal}, \cdot) = 0$

Loop for each episode:

    Initialize  $S$

    Loop for each step of episode:

        Choose  $A$  from  $S$  using policy derived from  $Q$  (e.g.,  $\varepsilon$ -greedy)

        Take action  $A$ , observe  $R, S'$

$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma \max_a Q(S', a) - Q(S, A)]$

$S \leftarrow S'$

    until  $S$  is terminal



## Other topics

---

See RL Book:

<http://incompleteideas.net/book/RLbook2020.pdf>